THE COMPLETE

RUST
PROGRAMMING GUIDE

From Installation to Mastery

Every Concept • Every Syntax • Every Detail

A Comprehensive Reference & Tutorial

Table of Contents

CHAPTER 1

Introduction to Rust

What is Rust?

Rust is a systems programming language focused on safety, speed, and concurrency. It accomplishes these goals without needing a garbage collector, making it useful for a number of use cases other languages are not well-suited for: embedding in other languages, programs with specific space and time requirements, and writing low-level code, such as device drivers and operating systems.

Rust was originally designed by Graydon Hoare at Mozilla Research, with contributions from many others. It has consistently been voted the 'most loved programming language' in developer surveys. The Rust Foundation now oversees the language's development.

Why Learn Rust?

- **Memory Safety Without GC** — Rust guarantees memory safety at compile time through its ownership system, without needing a garbage collector.
- **Zero-Cost Abstractions** — High-level features compile down to code as efficient as hand-written low-level code.
- **Fearless Concurrency** — The type system prevents data races at compile time.
- **Rich Type System** — Algebraic data types, pattern matching, traits, and generics.
- **Excellent Tooling** — Cargo (build system + package manager), rustfmt (formatter), clippy (linter), rust-analyzer (IDE support).
- **Growing Ecosystem** — Used by Mozilla, Google, Microsoft, Amazon, Meta, Dropbox, Cloudflare, and many more.
- **Cross-Platform** — Compiles to native code on Windows, macOS, Linux, and many embedded targets.

What Can You Build with Rust?

- Operating systems and kernels (Linux kernel modules, Redox OS)
- Web servers and APIs (Actix-web, Axum, Rocket)
- Command-line tools (ripgrep, fd, bat, exa)
- WebAssembly applications
- Game engines (Bevy, Amethyst)
- Embedded systems and IoT devices
- Blockchain and cryptocurrency systems
- Database engines (TiKV, SurrealDB)
- Networking tools and protocols

CHAPTER 2

Installation & Setup

Installing Rust with rustup

The recommended way to install Rust is through **rustup**, the official Rust toolchain installer. It manages Rust versions and associated tools.

On Linux / macOS

Terminal

```
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

After installation, add Rust to your PATH:

Terminal

```
source $HOME/.cargo/env
```

On Windows

Download and run **rustup-init.exe** from https://rustup.rs. You will also need the Microsoft C++ Build Tools (Visual Studio installer).

Verify Installation

Terminal

```
# Check Rust compiler version
rustc --version
# Output: rustc 1.XX.0 (hash date)

# Check Cargo version
cargo --version
# Output: cargo 1.XX.0 (hash date)

# Check rustup version
rustup --version
```

Managing Toolchains

rustup Commands

```
# Update Rust to latest stable
rustup update

# Install nightly toolchain
rustup install nightly

# Switch default toolchain
rustup default nightly
rustup default stable

# Use nightly for a specific project
rustup override set nightly

# List installed toolchains
rustup toolchain list

# Add a compilation target (e.g., WebAssembly)
rustup target add wasm32-unknown-unknown
```

Essential Components

Components

```
# Install the formatter
rustup component add rustfmt

# Install the linter
rustup component add clippy

# Install rust-analyzer (IDE support)
rustup component add rust-analyzer

# Install documentation locally
rustup component add rust-docs
rustup doc  # Opens local docs in browser
```

IDE / Editor Setup

- **VS Code** — Install the 'rust-analyzer' extension. It provides autocomplete, inline type hints, error highlighting, and code actions.
- **IntelliJ / CLion** — Install the 'Rust' plugin by JetBrains.
- **Neovim** — Use rust-analyzer with nvim-lspconfig.
- **Sublime Text** — Install the LSP and LSP-rust-analyzer packages.

CHAPTER 3

Hello World & Cargo

Your First Rust Program (without Cargo)

Create a file called **main.rs**:

main.rs

```
fn main() {
    println!("Hello, World!");
}
```

Compile and run:

Terminal

```
$ rustc main.rs
$ ./main
Hello, World!
```

Anatomy of a Rust Program

- **fn main()** — The entry point of every Rust executable. The **fn** keyword declares a function.
- **println!()** — A macro (note the **!**) that prints text to standard output with a newline.
- **Semicolons** — Statements end with a semicolon. Expressions (the last line in a block) may omit it to return a value.
- **Curly braces { }** — Define the function body / block scope.

Introduction to Cargo

Cargo is Rust's build system and package manager. It handles downloading dependencies, compiling packages, making distributable packages, and uploading them to crates.io (the Rust package registry).

Creating a New Project

Terminal

```
# Create a new binary project
cargo new hello_cargo
cd hello_cargo

# Or create a library project
cargo new my_library --lib

# Initialize in existing directory
cargo init
```

Project Structure

Project Layout

```
hello_cargo/
  Cargo.toml       # Project manifest (dependencies, metadata)
  src/
    main.rs         # Entry point for binary crate
    lib.rs          # Entry point for library crate (if --lib)
  target/           # Build output directory (auto-generated)
  .gitignore        # Git ignore file (auto-generated)
```

Cargo.toml Explained

Cargo.toml

```
[package]
name = "hello_cargo"
version = "0.1.0"
edition = "2021"           # Rust edition (2015, 2018, 2021)
authors = ["Your Name <email@example.com>"]
description = "A sample project"
license = "MIT"

[dependencies]
serde = "1.0"              # Add external crates here
serde_json = "1.0"
tokio = { version = "1", features = ["full"] }

[dev-dependencies]
criterion = "0.5"          # Only for tests/benchmarks

[[bin]]
name = "hello_cargo"
path = "src/main.rs"
```

Essential Cargo Commands

Cargo Commands

```
# Build the project (debug mode)
cargo build

# Build and run
cargo run

# Build with optimizations (release mode)
cargo build --release

# Check code compiles without building
cargo check

# Run tests
cargo test

# Generate and open documentation
cargo doc --open

# Format code
cargo fmt

# Run the linter
cargo clippy

# Update dependencies
cargo update

# Clean build artifacts
cargo clean

# Publish to crates.io
cargo publish
```

CHAPTER 4

Variables & Mutability

Immutable by Default

In Rust, variables are **immutable by default**. This is one of Rust's key design decisions that encourages you to write code in a way that takes advantage of the safety and easy concurrency that Rust offers.

Immutable Variables

```
fn main() {
    let x = 5;
    println!("x = {}", x);

    // x = 6;  // ERROR! Cannot assign twice to immutable variable
}
```

Mutable Variables

Add the **mut** keyword to make a variable mutable:

Mutable Variables

```
fn main() {
    let mut x = 5;
    println!("x = {}", x);

    x = 6;  // This is fine!
    println!("x = {}", x);
}
```

Constants

Constants are always immutable, must have a type annotation, and can be declared in any scope including the global scope. They must be set to a constant expression, not a runtime value.

Constants

```
// Constants use SCREAMING_SNAKE_CASE by convention
const MAX_POINTS: u32 = 100_000;
const PI: f64 = 3.14159265358979;
const GREETING: &str = "Hello";

fn main() {
    println!("Max: {}", MAX_POINTS);
}
```

Shadowing

Rust allows you to declare a new variable with the same name as a previous variable. The new variable **shadows** the previous one. This is different from **mut** because you can change the type and the variable remains immutable.

Shadowing

```
fn main() {
    let x = 5;
    let x = x + 1;       // x is now 6 (shadowed)
    let x = x * 2;       // x is now 12 (shadowed again)
    println!("x = {}", x);

    // Shadowing can change the type!
    let spaces = "   ";          // &str
    let spaces = spaces.len();   // usize -- different type, same name
    println!("spaces = {}", spaces);

    // With mut, you CANNOT change the type:
    // let mut s = "hello";
    // s = s.len();  // ERROR: mismatched types
}
```

Static Variables

Static Variables

```
// Static variables have a fixed address in memory
static LANGUAGE: &str = "Rust";
static mut COUNTER: u32 = 0;  // Mutable statics require unsafe

fn main() {
    println!("Language: {}", LANGUAGE);

    // Accessing mutable statics is unsafe
    unsafe {
        COUNTER += 1;
        println!("Counter: {}", COUNTER);
    }
}
```

NOTE

Mutable static variables are unsafe because they can cause data races in concurrent code.
Prefer using thread-safe types like Mutex or Atomic types instead.

CHAPTER 5

Data Types

Every value in Rust has a specific **data type**. Rust is a **statically typed** language, meaning it must know the types of all variables at compile time. The compiler can usually infer the type, but sometimes you need to add a type annotation.

Scalar Types

Scalar types represent a single value. Rust has four primary scalar types.

1. Integer Types

Length	Signed	Unsigned	Range (Signed)
8-bit	i8	u8	-128 to 127
16-bit	i16	u16	-32,768 to 32,767
32-bit	i32 (default)	u32	-2B to 2B
64-bit	i64	u64	Very large
128-bit	i128	u128	Extremely large
arch	isize	usize	Pointer-sized

Integer Literals

Integer Literals

```
let decimal     = 98_222;      // Underscores for readability
let hex         = 0xff;        // Hexadecimal
let octal       = 0o77;        // Octal
let binary      = 0b1111_0000; // Binary
let byte        = b'A';        // Byte (u8 only)
let typed: i64  = 42i64;       // Type suffix
let typed2: u8  = 255u8;
```

2. Floating-Point Types

Floating-Point

```
fn main() {
    let x = 2.0;       // f64 (default, double precision)
    let y: f32 = 3.0;  // f32 (single precision)

    // Arithmetic
    let sum = 5.0 + 10.0;
    let difference = 95.5 - 4.3;
    let product = 4.0 * 30.0;
    let quotient = 56.7 / 32.2;
    let truncated = -5 / 3;      // Results in -1 (integer division)
    let remainder = 43 % 5;      // Results in 3
}
```

3. Boolean Type

Boolean

```
fn main() {
    let t = true;
    let f: bool = false;  // With explicit type annotation

    // Booleans are one byte in size
    println!("Size of bool: {} byte", std::mem::size_of::<bool>());
}
```

4. Character Type

Character

```
fn main() {
    let c = 'z';
    let z: char = 'Z';
    let heart = '\u{2764}';  // Unicode: heart emoji
    let crab = '\u{1F980}';  // Unicode: crab emoji

    // char is 4 bytes (Unicode Scalar Value)
    println!("Size of char: {} bytes", std::mem::size_of::<char>());
}
```

TIP

Rust's char type is 4 bytes and represents a Unicode Scalar Value, which means it can repr
esent much more than just ASCII -- accented letters, Chinese characters, emoji, and zero-w
idth spaces are all valid char values.

Compound Types

Tuples

Tuples

```
fn main() {
    // Tuples group multiple values of different types
    let tup: (i32, f64, u8) = (500, 6.4, 1);

    // Destructuring
    let (x, y, z) = tup;
    println!("y = {}", y);

    // Access by index
    let five_hundred = tup.0;
    let six_point_four = tup.1;
    let one = tup.2;

    // Unit tuple (empty tuple)
    let unit: () = ();
}
```

Arrays

Arrays

```
fn main() {
    // Fixed-size array (stack-allocated)
    let a = [1, 2, 3, 4, 5];

    // With type annotation: [type; length]
    let a: [i32; 5] = [1, 2, 3, 4, 5];

    // Initialize with same value: [value; length]
    let a = [3; 5];  // Same as [3, 3, 3, 3, 3]

    // Access elements
    let first = a[0];
    let second = a[1];

    // Slices
    let slice = &a[1..3];  // &[i32] -- elements at index 1 and 2

    // Length
    println!("Length: {}", a.len());

    // Iterate
    for element in a.iter() {
        println!("{}", element);
    }
}
```

NOTE

Array index out of bounds will cause a runtime panic in Rust, not undefined behavior like in C/C++. Rust checks array bounds at runtime.

Type Aliases

Type Aliases

```
type Kilometers = i32;
type Thunk = Box<dyn Fn() + Send + 'static>;

fn main() {
    let x: Kilometers = 5;
    let y: i32 = 10;
    // x and y are the same type!
    println!("{}", x + y);  // Works fine
}
```

Type Casting

Type Casting with as

```
fn main() {
    let x: i32 = 42;
    let y: f64 = x as f64;        // i32 -> f64
    let z: u8 = 255;
    let w: i8 = z as i8;          // 255 -> -1 (wraps)

    let c = 'A';
    let num = c as u32;           // char -> u32 (65)

    let float = 65.99_f64;
    let int = float as u8;        // Truncates to 65
}
```

CHAPTER 6

Functions

Function Syntax

Functions

```rust
// Basic function
fn greet() {
    println!("Hello!");
}

// Function with parameters
fn greet_person(name: &str) {
    println!("Hello, {}!", name);
}

// Function with return value
fn add(a: i32, b: i32) -> i32 {
    a + b  // No semicolon = implicit return (expression)
}

// Explicit return
fn absolute(x: i32) -> i32 {
    if x < 0 {
        return -x;  // Early return
    }
    x  // Implicit return
}

fn main() {
    greet();
    greet_person("Alice");
    let result = add(5, 3);
    println!("5 + 3 = {}", result);
}
```

Statements vs Expressions

This is a crucial Rust concept. **Statements** perform actions but don't return values. **Expressions** evaluate to a value. In Rust, most things are expressions.

Statements vs Expressions

```
fn main() {
    // Statement: let binding does NOT return a value
    let x = 5;  // Statement
    // let y = (let z = 6);  // ERROR! let is a statement

    // Expression: block {} evaluates to its last expression
    let y = {
        let x = 3;
        x + 1      // No semicolon! This is an expression
    };  // y = 4

    // Adding a semicolon turns an expression into a statement
    let z = {
        let x = 3;
        x + 1;     // Semicolon! Returns () (unit type)
    };  // z = ()

    println!("y = {}", y);  // 4
}
```

TIP

If a function body ends with a semicolon on the last line, it returns () (the unit type).
This is the most common Rust beginner mistake for functions that should return a value.

Functions with Multiple Return Values

Multiple Return Values

```
// Return a tuple
fn swap(a: i32, b: i32) -> (i32, i32) {
    (b, a)
}

// Destructure the return value
fn main() {
    let (x, y) = swap(1, 2);
    println!("x={}, y={}", x, y);  // x=2, y=1
}
```

Diverging Functions (Never Type)

Diverging Functions

```rust
// Functions that never return use ! (the never type)
fn forever() -> ! {
    loop {
        // Runs forever
    }
}

fn crash() -> ! {
    panic!("This function never returns");
}
```

CHAPTER 7

Control Flow

if Expressions

if Expressions

```
fn main() {
    let number = 7;

    // Basic if/else
    if number < 5 {
        println!("less than 5");
    } else if number < 10 {
        println!("between 5 and 9");
    } else {
        println!("10 or greater");
    }

    // if is an EXPRESSION -- can be used in let
    let condition = true;
    let value = if condition { 5 } else { 6 };
    println!("value = {}", value);

    // Both branches must return the same type!
    // let bad = if true { 5 } else { "six" }; // ERROR
}
```

NOTE

Rust does NOT automatically convert non-boolean types to boolean. The condition must be a bool. 'if number { }' will not compile -- use 'if number != 0 { }' instead.

Loops

loop (Infinite Loop)

loop

```
fn main() {
    let mut counter = 0;

    // loop can return a value with break
    let result = loop {
        counter += 1;
        if counter == 10 {
            break counter * 2;  // Returns 20
        }
    };
    println!("result = {}", result);

    // Loop labels for nested loops
    'outer: loop {
        'inner: loop {
            break 'outer;  // Breaks the outer loop
        }
    }
}
```

while Loop

while

```
fn main() {
    let mut number = 3;
    while number != 0 {
        println!("{}!", number);
        number -= 1;
    }
    println!("LIFTOFF!");
}
```

for Loop

for Loops

```
fn main() {
    // Iterate over a collection
    let a = [10, 20, 30, 40, 50];
    for element in a.iter() {
        println!("{}", element);
    }

    // Range (most common)
    for i in 0..5 {
        println!("{}", i);  // 0, 1, 2, 3, 4
    }

    // Inclusive range
    for i in 0..=5 {
        println!("{}", i);  // 0, 1, 2, 3, 4, 5
    }

    // Reverse
    for i in (1..4).rev() {
        println!("{}", i);  // 3, 2, 1
    }

    // Enumerate (index + value)
    let names = vec!["Alice", "Bob", "Charlie"];
    for (i, name) in names.iter().enumerate() {
        println!("{}: {}", i, name);
    }
}
```

while let

while let

```
fn main() {
    let mut stack = vec![1, 2, 3];

    // Loop while pattern matches
    while let Some(top) = stack.pop() {
        println!("{}", top);
    }  // Prints: 3, 2, 1
}
```

if let

if let

```
fn main() {
    let some_value: Option<i32> = Some(42);

    // Instead of a verbose match
    if let Some(val) = some_value {
        println!("Got: {}", val);
    } else {
        println!("Got nothing");
    }
}
```

CHAPTER 8 Ownership & Borrowing

Ownership is Rust's most unique feature and is fundamental to understanding the language. It enables Rust to make memory safety guarantees without needing a garbage collector.

Ownership Rules

- **Rule 1:** Each value in Rust has a variable that is called its **owner**.
- **Rule 2:** There can only be **one owner at a time**.
- **Rule 3:** When the owner goes out of scope, the value is **dropped** (freed).

Move Semantics

Move Semantics

```
fn main() {
    // For heap-allocated data, assignment MOVES ownership
    let s1 = String::from("hello");
    let s2 = s1;  // s1 is MOVED to s2

    // println!("{}", s1);  // ERROR! s1 is no longer valid
    println!("{}", s2);     // OK

    // For stack types (Copy trait), values are COPIED
    let x = 5;
    let y = x;  // x is copied, both are valid
    println!("x={}, y={}", x, y);  // Both work
}
```

Clone (Deep Copy)

Clone

```
fn main() {
    let s1 = String::from("hello");
    let s2 = s1.clone();  // Deep copy of heap data

    println!("s1={}, s2={}", s1, s2);  // Both valid!
}
```

Ownership and Functions

Ownership and Functions

```rust
fn takes_ownership(s: String) {
    println!("{}", s);
}  // s is dropped here

fn makes_copy(x: i32) {
    println!("{}", x);
}  // x goes out of scope, nothing special

fn gives_ownership() -> String {
    String::from("hello")  // Ownership moves to caller
}

fn takes_and_gives_back(s: String) -> String {
    s  // Ownership moves back to caller
}

fn main() {
    let s = String::from("hello");
    takes_ownership(s);  // s moved into function
    // println!("{}", s);  // ERROR! s is no longer valid

    let x = 5;
    makes_copy(x);  // i32 is Copy, x still valid
    println!("{}", x);  // OK

    let s1 = gives_ownership();  // Gets ownership
    let s2 = takes_and_gives_back(s1);  // s1 moved, s2 gets it back
}
```

References & Borrowing

Instead of transferring ownership, you can **borrow** a value by creating a **reference**. References allow you to refer to a value without taking ownership.

References

```
// Immutable reference (&T)
fn calculate_length(s: &String) -> usize {
    s.len()
}  // s goes out of scope, but it doesn't own the String

// Mutable reference (&mut T)
fn change(s: &mut String) {
    s.push_str(", world!");
}

fn main() {
    let s1 = String::from("hello");
    let len = calculate_length(&s1);  // Borrow s1
    println!("Length of '{}' is {}", s1, len);  // s1 still valid!

    let mut s2 = String::from("hello");
    change(&mut s2);  // Mutable borrow
    println!("{}", s2);  // "hello, world!"
}
```

Borrowing Rules

- **Rule 1:** At any given time, you can have EITHER one mutable reference OR any number of immutable references.
- **Rule 2:** References must always be valid (no dangling references).

Borrowing Rules

```
fn main() {
    let mut s = String::from("hello");

    // Multiple immutable borrows: OK
    let r1 = &s;
    let r2 = &s;
    println!("{} and {}", r1, r2);
    // r1 and r2 are no longer used after this point

    // Mutable borrow after immutable borrows are done: OK
    let r3 = &mut s;
    println!("{}", r3);

    // CANNOT have mutable + immutable at the same time:
    // let r4 = &s;
    // let r5 = &mut s;
    // println!("{}, {}", r4, r5);  // ERROR!
}
```

The Slice Type

Slices

```
fn first_word(s: &str) -> &str {
    let bytes = s.as_bytes();
    for (i, &item) in bytes.iter().enumerate() {
        if item == b' ' {
            return &s[0..i];
        }
    }
    &s[..]
}

fn main() {
    let s = String::from("hello world");

    // String slices
    let hello = &s[0..5];    // "hello"
    let world = &s[6..11];   // "world"
    let hello2 = &s[..5];    // Same as 0..5
    let world2 = &s[6..];    // Same as 6..len
    let whole = &s[..];      // Entire string

    // Array slices
    let a = [1, 2, 3, 4, 5];
    let slice = &a[1..3];    // &[i32] = [2, 3]

    let word = first_word(&s);
    println!("First word: {}", word);
}
```

CHAPTER 9

Structs

Defining and Instantiating Structs

Structs

```
// Define a struct
struct User {
    username: String,
    email: String,
    sign_in_count: u64,
    active: bool,
}

fn main() {
    // Create an instance
    let user1 = User {
        email: String::from("alice@example.com"),
        username: String::from("alice"),
        active: true,
        sign_in_count: 1,
    };

    // Access fields with dot notation
    println!("Email: {}", user1.email);

    // Mutable instance (entire struct must be mutable)
    let mut user2 = User {
        email: String::from("bob@example.com"),
        username: String::from("bob"),
        active: true,
        sign_in_count: 1,
    };
    user2.email = String::from("bob_new@example.com");
}
```

Field Init Shorthand & Struct Update

Field Init & Update Syntax

```
fn build_user(email: String, username: String) -> User {
    User {
        email,           // Field init shorthand
        username,        // Same name = shorthand
        active: true,
        sign_in_count: 1,
    }
}

fn main() {
    let user1 = build_user(
        String::from("a@b.com"),
        String::from("alice"),
    );

    // Struct update syntax (like spread in JS)
    let user2 = User {
        email: String::from("new@b.com"),
        ..user1  // Fill remaining fields from user1
    };
    // Note: user1.username was MOVED to user2
}
```

Tuple Structs & Unit Structs

Tuple & Unit Structs

```
// Tuple structs -- named tuples
struct Color(i32, i32, i32);
struct Point(f64, f64, f64);

fn main() {
    let black = Color(0, 0, 0);
    let origin = Point(0.0, 0.0, 0.0);

    println!("Red: {}", black.0);
    println!("X: {}", origin.0);
}

// Unit-like struct (no fields)
struct AlwaysEqual;
let subject = AlwaysEqual;
```

Methods with impl

Methods

```
#[derive(Debug)]
struct Rectangle {
    width: f64,
    height: f64,
}

impl Rectangle {
    // Method (takes &self)
    fn area(&self) -> f64 {
        self.width * self.height
    }

    fn perimeter(&self) -> f64 {
        2.0 * (self.width + self.height)
    }

    // Method that mutates self
    fn scale(&mut self, factor: f64) {
        self.width *= factor;
        self.height *= factor;
    }

    // Method that takes ownership
    fn into_square(self) -> Rectangle {
        let side = self.width.max(self.height);
        Rectangle { width: side, height: side }
    }

    // Associated function (no self) -- like a static method
    fn square(size: f64) -> Rectangle {
        Rectangle { width: size, height: size }
    }

    fn new(width: f64, height: f64) -> Self {
        Self { width, height }  // Self refers to Rectangle
    }
}

fn main() {
    let mut rect = Rectangle::new(30.0, 50.0);
    println!("Area: {}", rect.area());
    println!("Debug: {:?}", rect);

    rect.scale(2.0);
    let sq = Rectangle::square(10.0);  // Associated function
}
```

Derive Macros

Derive Macros

```
// Common derive macros
#[derive(Debug)]       // Enable {:?} formatting
#[derive(Clone)]       // Enable .clone()
#[derive(Copy, Clone)] // Enable implicit copying
#[derive(PartialEq)]   // Enable == and !=
#[derive(Eq)]          // Enable full equality (requires PartialEq)
#[derive(Hash)]        // Enable use as HashMap key
#[derive(Default)]     // Enable Type::default()
#[derive(PartialOrd)]  // Enable < > <= >=
#[derive(Ord)]         // Enable total ordering

// Combine multiple derives
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
struct Point {
    x: i32,
    y: i32,
}
```

CHAPTER 10

Enums & Pattern Matching

Defining Enums

Enums

```
// Simple enum
enum Direction {
    North,
    South,
    East,
    West,
}

// Enums with data (each variant can hold different types!)
enum Message {
    Quit,                       // No data
    Move { x: i32, y: i32 },    // Named fields (like struct)
    Write(String),              // Single String
    ChangeColor(i32, i32, i32), // Three i32 values
}

// Enums can have methods too
impl Message {
    fn call(&self) {
        // method body
    }
}

fn main() {
    let dir = Direction::North;
    let msg = Message::Write(String::from("hello"));
    msg.call();
}
```

Option -- Null Safety

Rust does NOT have null. Instead, it uses the **Option<T>** enum to represent a value that might or might not exist. This is defined in the standard library:

Option<T>

```rust
// Definition (already in std)
enum Option<T> {
    Some(T),
    None,
}

fn main() {
    let some_number: Option<i32> = Some(5);
    let absent_number: Option<i32> = None;

    // You CANNOT use Option<T> as T directly
    // let sum = some_number + 5;  // ERROR!

    // You must unwrap or match
    let value = some_number.unwrap();       // Panics if None
    let value = some_number.unwrap_or(0);   // Default if None
    let value = some_number.expect("msg");  // Panics with message

    // Safe check
    if some_number.is_some() {
        println!("Has value: {}", some_number.unwrap());
    }

    // Map/and_then (functional style)
    let doubled = some_number.map(|x| x * 2);  // Some(10)
}
```

The match Expression

match Expression

```
enum Coin {
    Penny,
    Nickel,
    Dime,
    Quarter(UsState),
}

#[derive(Debug)]
enum UsState {
    Alabama, Alaska, Arizona, /*...*/
}

fn value_in_cents(coin: &Coin) -> u8 {
    match coin {
        Coin::Penny => {
            println!("Lucky penny!");
            1
        }
        Coin::Nickel => 5,
        Coin::Dime => 10,
        Coin::Quarter(state) => {
            println!("Quarter from {:?}", state);
            25
        }
    }
}
```

Pattern Matching Features

Advanced Pattern Matching

```rust
fn main() {
    let x = 5;

    // Match with multiple patterns
    match x {
        1 | 2 => println!("one or two"),
        3..=5 => println!("three to five"),  // Range pattern
        _ => println!("something else"),      // Catch-all
    }

    // Match with guards
    let num = Some(4);
    match num {
        Some(x) if x < 5 => println!("less than five: {}", x),
        Some(x) => println!("{}", x),
        None => (),
    }

    // Destructuring structs in match
    struct Point { x: i32, y: i32 }
    let p = Point { x: 0, y: 7 };
    match p {
        Point { x: 0, y } => println!("On y-axis at {}", y),
        Point { x, y: 0 } => println!("On x-axis at {}", x),
        Point { x, y } => println!("({}, {})", x, y),
    }

    // @ bindings
    let msg = Message::Write(String::from("hi"));
    match msg {
        Message::Write(text @ _) => println!("Text: {}", text),
        _ => {}
    }
}
```

CHAPTER 11

Collections

Vec -- Vectors

Vectors

```rust
fn main() {
    // Creating vectors
    let v1: Vec<i32> = Vec::new();          // Empty
    let v2 = vec![1, 2, 3];                 // With vec! macro
    let v3 = vec![0; 10];                   // 10 zeros

    // Adding elements
    let mut v = Vec::new();
    v.push(1);
    v.push(2);
    v.push(3);

    // Accessing elements
    let third = &v[2];                      // Panics if out of bounds
    let third = v.get(2);                   // Returns Option<&T>

    match v.get(100) {
        Some(val) => println!("{}", val),
        None => println!("Index out of bounds"),
    }

    // Iterating
    for i in &v {
        println!("{}", i);
    }

    // Mutating while iterating
    for i in &mut v {
        *i += 10;
    }

    // Useful methods
    v.len();                // Length
    v.is_empty();           // Check empty
    v.contains(&1);         // Check contains
    v.pop();                // Remove last (returns Option)
    v.remove(0);            // Remove at index
    v.insert(0, 99);        // Insert at index
    v.sort();               // Sort in place
    v.dedup();              // Remove consecutive duplicates
    v.retain(|&x| x > 5);  // Keep only matching
    v.truncate(3);          // Keep first 3 elements
    v.clear();              // Remove all
}
```

String

String

```
fn main() {
    // Creating strings
    let s1 = String::new();                  // Empty
    let s2 = String::from("hello");          // From &str
    let s3 = "hello".to_string();            // Also from &str
    let s4 = format!("{}-{}", "hello", "world"); // Format macro

    // Appending
    let mut s = String::from("hello");
    s.push(' ');                  // Push single char
    s.push_str("world");          // Push string slice

    // Concatenation
    let s1 = String::from("Hello, ");
    let s2 = String::from("world!");
    let s3 = s1 + &s2;  // s1 is MOVED, s2 is borrowed
    // s1 no longer valid!

    // Iteration
    for c in "hello".chars() {
        println!("{}", c);  // h, e, l, l, o
    }
    for b in "hello".bytes() {
        println!("{}", b);  // 104, 101, ...
    }

    // Useful methods
    let s = String::from("  Hello, World!  ");
    s.len();                     // Byte length
    s.is_empty();                // Check empty
    s.contains("World");         // Search
    s.starts_with("  H");        // Prefix check
    s.ends_with("!  ");          // Suffix check
    s.trim();                    // Remove whitespace
    s.to_uppercase();            // "  HELLO, WORLD!  "
    s.to_lowercase();            // "  hello, world!  "
    s.replace("World", "Rust");  // Replace
    s.split(',');                // Split iterator
}
```

TIP

Rust strings are UTF-8 encoded. You cannot index into a String with s[0] because UTF-8 cha
racters can be multiple bytes. Use .chars().nth(0) or slicing with care.

HashMap

HashMap

```rust
use std::collections::HashMap;

fn main() {
    // Creating
    let mut scores = HashMap::new();
    scores.insert(String::from("Blue"), 10);
    scores.insert(String::from("Red"), 50);

    // From iterators
    let teams = vec!["Blue", "Red"];
    let initial = vec![10, 50];
    let scores: HashMap<_, _> =
        teams.into_iter().zip(initial.into_iter()).collect();

    // Accessing
    let score = scores.get("Blue");  // Returns Option<&V>

    // Iterating
    for (key, value) in &scores {
        println!("{}: {}", key, value);
    }

    // Update patterns
    let mut map = HashMap::new();
    map.insert("key", 1);       // Insert
    map.insert("key", 2);       // Overwrite

    // Insert only if key doesn't exist
    map.entry("key").or_insert(99);   // Won't overwrite
    map.entry("new").or_insert(99);   // Will insert

    // Update based on old value (word counter)
    let text = "hello world hello";
    let mut word_count = HashMap::new();
    for word in text.split_whitespace() {
        let count = word_count.entry(word).or_insert(0);
        *count += 1;
    }
    // {"hello": 2, "world": 1}
}
```

Other Collections

- **HashSet<T>** — A set of unique values. Uses HashMap internally.
- **BTreeMap<K, V>** — Sorted map (by key). O(log n) operations.
- **BTreeSet<T>** — Sorted set.
- **VecDeque<T>** — Double-ended queue. Efficient push/pop from both ends.
- **LinkedList<T>** — Doubly-linked list (rarely needed).
- **BinaryHeap<T>** — Max-heap / priority queue.

CHAPTER 12

Error Handling

Unrecoverable Errors: panic!

panic!

```
fn main() {
    // Explicit panic
    // panic!("crash and burn");

    // Implicit panic (index out of bounds)
    let v = vec![1, 2, 3];
    // v[99];  // This will panic!

    // Set RUST_BACKTRACE=1 for detailed panic info
}
```

Recoverable Errors: Result

Result<T, E>

```
// Result is defined in std
enum Result<T, E> {
    Ok(T),
    Err(E),
}

use std::fs::File;
use std::io::{self, Read};

fn main() {
    // Opening a file returns Result
    let file = File::open("hello.txt");

    let file = match file {
        Ok(f) => f,
        Err(error) => match error.kind() {
            io::ErrorKind::NotFound => {
                match File::create("hello.txt") {
                    Ok(fc) => fc,
                    Err(e) => panic!("Cant create: {:?}", e),
                }
            }
            other => panic!("Problem opening: {:?}", other),
        },
    };
}
```

Shortcuts: unwrap, expect, and ?

Error Shortcuts

```
use std::fs::File;
use std::io::{self, Read};

// unwrap: panics on Err
let f = File::open("hello.txt").unwrap();

// expect: panics with custom message
let f = File::open("hello.txt")
    .expect("Failed to open hello.txt");

// ? operator: returns the error to the caller
fn read_username() -> Result<String, io::Error> {
    let mut f = File::open("hello.txt")?;  // Returns Err if fails
    let mut s = String::new();
    f.read_to_string(&mut s)?;             // Returns Err if fails
    Ok(s)
}

// Chaining with ?
fn read_username_short() -> Result<String, io::Error> {
    let mut s = String::new();
    File::open("hello.txt")?.read_to_string(&mut s)?;
    Ok(s)
}

// Even shorter: std::fs has a convenience function
fn read_username_shortest() -> Result<String, io::Error> {
    std::fs::read_to_string("hello.txt")
}
```

Custom Error Types

Custom Error Types

```
use std::fmt;

#[derive(Debug)]
enum AppError {
    IoError(std::io::Error),
    ParseError(std::num::ParseIntError),
    Custom(String),
}

impl fmt::Display for AppError {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            AppError::IoError(e) => write!(f, "IO error: {}", e),
            AppError::ParseError(e) => write!(f, "Parse: {}", e),
            AppError::Custom(msg) => write!(f, "{}", msg),
        }
    }
}

// Enable ? to convert std::io::Error -> AppError
impl From<std::io::Error> for AppError {
    fn from(e: std::io::Error) -> Self {
        AppError::IoError(e)
    }
}

impl From<std::num::ParseIntError> for AppError {
    fn from(e: std::num::ParseIntError) -> Self {
        AppError::ParseError(e)
    }
}

fn do_stuff() -> Result<i32, AppError> {
    let content = std::fs::read_to_string("num.txt")?;
    let num: i32 = content.trim().parse()?;
    Ok(num)
}
```

TIP

For real-world projects, consider using the 'thiserror' crate for deriving error types, or 'anyhow' crate for flexible error handling in applications.

CHAPTER 13

Generics

Generic Functions

Generic Functions

```
// Generic function
fn largest<T: PartialOrd>(list: &[T]) -> &T {
    let mut largest = &list[0];
    for item in list {
        if item > largest {
            largest = item;
        }
    }
    largest
}

fn main() {
    let numbers = vec![34, 50, 25, 100, 65];
    println!("Largest: {}", largest(&numbers));

    let chars = vec!['y', 'm', 'a', 'q'];
    println!("Largest: {}", largest(&chars));
}
```

Generic Structs

Generic Structs

```rust
// Generic struct with one type
struct Point<T> {
    x: T,
    y: T,
}

// Multiple generic types
struct Point2<T, U> {
    x: T,
    y: U,
}

// Methods on generic structs
impl<T> Point<T> {
    fn x(&self) -> &T {
        &self.x
    }
}

// Methods only for specific types
impl Point<f64> {
    fn distance_from_origin(&self) -> f64 {
        (self.x.powi(2) + self.y.powi(2)).sqrt()
    }
}

fn main() {
    let int_point = Point { x: 5, y: 10 };
    let float_point = Point { x: 1.0, y: 4.0 };
    let mixed = Point2 { x: 5, y: 4.0 };
}
```

Generic Enums

Generic Enums

```rust
// Option and Result are generic enums!
enum Option<T> {
    Some(T),
    None,
}

enum Result<T, E> {
    Ok(T),
    Err(E),
}

// Custom generic enum
enum Either<L, R> {
    Left(L),
    Right(R),
}
```

TIP

Rust uses monomorphization at compile time, which means generic code is as fast as hand-wr
itten specific code. There is zero runtime cost for generics.

CHAPTER 14

Traits

Traits define shared behavior in an abstract way. They are similar to interfaces in other languages, but more powerful because they support default implementations and can be used as bounds on generics.

Defining and Implementing Traits

Traits

```
// Define a trait
trait Summary {
    // Required method (no body)
    fn summarize_author(&self) -> String;

    // Default method (has a body, can be overridden)
    fn summarize(&self) -> String {
        format!("(Read more from {}...)", self.summarize_author())
    }
}

struct NewsArticle {
    title: String,
    author: String,
    content: String,
}

impl Summary for NewsArticle {
    fn summarize_author(&self) -> String {
        self.author.clone()
    }

    fn summarize(&self) -> String {
        format!("{}, by {}", self.title, self.author)
    }
}

struct Tweet {
    username: String,
    content: String,
}

impl Summary for Tweet {
    fn summarize_author(&self) -> String {
        format!("@{}", self.username)
    }
    // Uses default summarize()
}
```

Trait Bounds

Trait Bounds

```rust
// Trait bound syntax
fn notify<T: Summary>(item: &T) {
    println!("Breaking: {}", item.summarize());
}

// impl Trait syntax (shorthand)
fn notify2(item: &impl Summary) {
    println!("Breaking: {}", item.summarize());
}

// Multiple bounds
fn notify3(item: &(impl Summary + std::fmt::Display)) {
    println!("{}", item);
}

// where clause (cleaner for complex bounds)
fn some_function<T, U>(t: &T, u: &U) -> i32
where
    T: Summary + Clone,
    U: Clone + std::fmt::Debug,
{
    42
}

// Return impl Trait
fn create_summarizable() -> impl Summary {
    Tweet {
        username: String::from("bot"),
        content: String::from("hello"),
    }
}
```

Common Standard Library Traits

Trait	Purpose	Key Method
Display	User-facing formatting	fmt(&self, f)
Debug	Debug formatting {:?}	#[derive(Debug)]
Clone	Deep copy	.clone()
Copy	Implicit bitwise copy	#[derive(Copy)]
PartialEq / Eq	Equality comparison	== !=
PartialOrd / Ord	Ordering comparison	< > <= >=
Hash	Hashing for HashMaps	.hash()
Default	Default value	Type::default()
From / Into	Type conversions	.into(), From::from()
Iterator	Iteration protocol	.next()
Drop	Destructor logic	drop(&mut self)

Deref	Custom dereference	*val

Trait Objects (Dynamic Dispatch)

Trait Objects

```
// Box<dyn Trait> allows different types behind one interface
fn get_items() -> Vec<Box<dyn Summary>> {
    vec![
        Box::new(NewsArticle {
            title: String::from("Title"),
            author: String::from("Author"),
            content: String::from("Content"),
        }),
        Box::new(Tweet {
            username: String::from("user"),
            content: String::from("tweet"),
        }),
    ]
}

// &dyn Trait also works for references
fn print_summary(item: &dyn Summary) {
    println!("{}", item.summarize());
}
```

CHAPTER 15 Lifetimes

Lifetimes are Rust's way of ensuring that references are valid for as long as they need to be. Every reference in Rust has a **lifetime**, which is the scope for which that reference is valid. Most of the time, lifetimes are inferred, just like types.

Lifetime Annotation Syntax

Lifetime Annotations

```
// Lifetime annotations don't change how long references live.
// They describe the relationship between lifetimes of references.

// This function says: the returned reference will be valid
// for as long as BOTH input references are valid.
fn longest<'a>(x: &'a str, y: &'a str) -> &'a str {
    if x.len() > y.len() {
        x
    } else {
        y
    }
}

fn main() {
    let string1 = String::from("long string");
    let result;
    {
        let string2 = String::from("xyz");
        result = longest(string1.as_str(), string2.as_str());
        println!("Longest: {}", result);  // OK here
    }
    // println!("{}", result);  // ERROR if result borrows string2
}
```

Lifetimes in Structs

Struct Lifetimes

```
// A struct that holds a reference must have a lifetime annotation
struct ImportantExcerpt<'a> {
    part: &'a str,
}

impl<'a> ImportantExcerpt<'a> {
    fn level(&self) -> i32 {
        3
    }

    // Lifetime elision: output lifetime = &self lifetime
    fn announce(&self, announcement: &str) -> &str {
        println!("Attention: {}", announcement);
        self.part
    }
}

fn main() {
    let novel = String::from("Call me Ishmael. Some years ago...");
    let first = novel.split('.').next().unwrap();
    let excerpt = ImportantExcerpt { part: first };
}
```

Lifetime Elision Rules

The compiler applies three rules to infer lifetimes so you don't always have to write them:

- **Rule 1:** Each parameter that is a reference gets its own lifetime parameter.
- **Rule 2:** If there is exactly one input lifetime parameter, it is assigned to all output lifetime parameters.
- **Rule 3:** If one of the parameters is **&self** or **&mut self**, the lifetime of self is assigned to all output lifetime parameters.

The 'static Lifetime

'static Lifetime

```
// 'static means the reference lives for the entire program
let s: &'static str = "I live forever!";
// String literals are always 'static

// In trait bounds
fn long_lived<T: std::fmt::Display + 'static>(t: T) {
    println!("{}", t);
}
```

NOTE

Don't use 'static as a fix for all lifetime errors. It means the data lives for the entire program, which is rarely what you actually need. Most lifetime issues are better solved by restructuring your code.

CHAPTER 16

Closures & Iterators

Closures

Closures

```
fn main() {
    // Closure syntax
    let add = |a, b| a + b;
    let result = add(5, 3);

    // With type annotations
    let add_typed = |a: i32, b: i32| -> i32 { a + b };

    // Multi-line closure
    let expensive = |num: u32| -> u32 {
        println!("calculating...");
        std::thread::sleep(std::time::Duration::from_secs(2));
        num
    };

    // Capturing variables from environment
    let x = 4;
    let equal_to_x = |z| z == x;  // Captures x by reference
    println!("{}", equal_to_x(4));  // true

    // Move semantics: force closure to take ownership
    let s = String::from("hello");
    let closure = move || {
        println!("{}", s);  // s is MOVED into closure
    };
    closure();
    // println!("{}", s);  // ERROR: s was moved
}
```

Closure Traits: Fn, FnMut, FnOnce

Closure Traits

```rust
// FnOnce: takes ownership, can be called once
fn consume<F: FnOnce()>(f: F) {
    f();
}

// FnMut: borrows mutably, can be called multiple times
fn mutate<F: FnMut()>(mut f: F) {
    f();
    f();
}

// Fn: borrows immutably, can be called multiple times
fn borrow<F: Fn() -> i32>(f: F) {
    println!("{}", f());
    println!("{}", f());
}
```

Iterators

Iterators

```
fn main() {
    let v = vec![1, 2, 3, 4, 5];

    // Basic iteration
    let v_iter = v.iter();      // Borrows
    let v_iter = v.into_iter(); // Takes ownership
    let v_iter = v.iter_mut();  // Borrows mutably

    // Iterator adaptors (lazy -- must consume)
    let v = vec![1, 2, 3, 4, 5];

    // map: transform each element
    let doubled: Vec<i32> = v.iter().map(|x| x * 2).collect();

    // filter: keep only matching elements
    let evens: Vec<&i32> = v.iter().filter(|x| **x % 2 == 0).collect();

    // chain
    let v2 = vec![6, 7, 8];
    let combined: Vec<&i32> = v.iter().chain(v2.iter()).collect();

    // enumerate
    for (i, val) in v.iter().enumerate() {
        println!("{}: {}", i, val);
    }

    // zip
    let names = vec!["Alice", "Bob"];
    let ages = vec![30, 25];
    let people: Vec<_> = names.iter().zip(ages.iter()).collect();

    // Consuming adaptors
    let sum: i32 = v.iter().sum();
    let product: i32 = v.iter().product();
    let count = v.iter().count();
    let max = v.iter().max();
    let min = v.iter().min();
    let any_even = v.iter().any(|x| x % 2 == 0);
    let all_pos = v.iter().all(|x| *x > 0);
    let found = v.iter().find(|&&x| x == 3);
    let pos = v.iter().position(|&x| x == 3);

    // Chaining multiple operations
    let result: i32 = v.iter()
        .filter(|&&x| x > 2)
        .map(|x| x * x)
        .sum();  // 9 + 16 + 25 = 50
}
```

Implementing Iterator

Custom Iterator

```
struct Counter {
    count: u32,
    max: u32,
}

impl Counter {
    fn new(max: u32) -> Counter {
        Counter { count: 0, max }
    }
}

impl Iterator for Counter {
    type Item = u32;

    fn next(&mut self) -> Option<Self::Item> {
        if self.count < self.max {
            self.count += 1;
            Some(self.count)
        } else {
            None
        }
    }
}

fn main() {
    let sum: u32 = Counter::new(5)
        .zip(Counter::new(5).skip(1))
        .map(|(a, b)| a * b)
        .filter(|x| x % 3 == 0)
        .sum();
}
```

CHAPTER 17

Smart Pointers

Box -- Heap Allocation

```
Box<T>

fn main() {
    // Box stores data on the heap
    let b = Box::new(5);
    println!("b = {}", b);  // Auto-dereferences

    // Use cases:
    // 1. Recursive types (unknown size at compile time)
    enum List {
        Cons(i32, Box<List>),
        Nil,
    }
    let list = List::Cons(1,
        Box::new(List::Cons(2,
            Box::new(List::Cons(3,
                Box::new(List::Nil))))));

    // 2. Large data you want to transfer ownership without copy
    let large = Box::new([0u8; 1_000_000]);
}
```

Rc -- Reference Counting

```
Rc<T>

use std::rc::Rc;

fn main() {
    // Rc allows multiple owners (single-threaded only)
    let a = Rc::new(String::from("hello"));
    println!("count: {}", Rc::strong_count(&a));  // 1

    let b = Rc::clone(&a);  // Increments reference count
    println!("count: {}", Rc::strong_count(&a));  // 2

    {
        let c = Rc::clone(&a);
        println!("count: {}", Rc::strong_count(&a));  // 3
    }  // c dropped

    println!("count: {}", Rc::strong_count(&a));  // 2
}
```

RefCell -- Interior Mutability

RefCell<T>

```
use std::cell::RefCell;

fn main() {
    // RefCell enforces borrowing rules at RUNTIME
    let data = RefCell::new(vec![1, 2, 3]);

    // Immutable borrow
    let borrowed = data.borrow();
    println!("{:?}", borrowed);
    drop(borrowed);  // Must drop before mutable borrow

    // Mutable borrow
    data.borrow_mut().push(4);
    println!("{:?}", data.borrow());
}

// Common pattern: Rc<RefCell<T>> for shared mutable data
use std::rc::Rc;
let shared = Rc::new(RefCell::new(42));
let clone1 = Rc::clone(&shared);
*clone1.borrow_mut() += 1;
```

Arc -- Thread-Safe Reference Counting

Arc<T>

```
use std::sync::Arc;
use std::thread;

fn main() {
    // Arc = Atomic Reference Counted (thread-safe Rc)
    let data = Arc::new(vec![1, 2, 3]);

    let handles: Vec<_> = (0..3).map(|i| {
        let data = Arc::clone(&data);
        thread::spawn(move || {
            println!("Thread {}: {:?}", i, data);
        })
    }).collect();

    for handle in handles {
        handle.join().unwrap();
    }
}
```

Cow -- Clone on Write

Cow<T>

```rust
use std::borrow::Cow;

fn maybe_modify(input: &str) -> Cow<str> {
    if input.contains("bad") {
        // Allocates a new String only when needed
        Cow::Owned(input.replace("bad", "good"))
    } else {
        // No allocation -- just borrows
        Cow::Borrowed(input)
    }
}
```

CHAPTER 18

Concurrency

Threads

Threads

```rust
use std::thread;
use std::time::Duration;

fn main() {
    // Spawn a new thread
    let handle = thread::spawn(|| {
        for i in 1..=5 {
            println!("spawned thread: {}", i);
            thread::sleep(Duration::from_millis(1));
        }
    });

    for i in 1..=3 {
        println!("main thread: {}", i);
        thread::sleep(Duration::from_millis(1));
    }

    // Wait for thread to finish
    handle.join().unwrap();

    // Moving data into a thread
    let v = vec![1, 2, 3];
    let handle = thread::spawn(move || {
        println!("Vector: {:?}", v);
    });
    handle.join().unwrap();
}
```

Message Passing with Channels

Channels

```
use std::sync::mpsc;  // multi-producer, single-consumer
use std::thread;

fn main() {
    let (tx, rx) = mpsc::channel();

    // Clone tx for multiple producers
    let tx2 = tx.clone();

    thread::spawn(move || {
        tx.send(String::from("hello from 1")).unwrap();
    });

    thread::spawn(move || {
        tx2.send(String::from("hello from 2")).unwrap();
    });

    // Receive messages
    for received in rx {
        println!("Got: {}", received);
    }
}
```

Shared State: Mutex

Mutex

```
use std::sync::{Arc, Mutex};
use std::thread;

fn main() {
    // Arc<Mutex<T>> = thread-safe shared mutable state
    let counter = Arc::new(Mutex::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        let handle = thread::spawn(move || {
            let mut num = counter.lock().unwrap();
            *num += 1;
        });
        handles.push(handle);
    }

    for handle in handles {
        handle.join().unwrap();
    }

    println!("Result: {}", *counter.lock().unwrap());  // 10
}
```

Atomic Types

Atomics

```
use std::sync::atomic::{AtomicU64, Ordering};
use std::sync::Arc;
use std::thread;

fn main() {
    let counter = Arc::new(AtomicU64::new(0));
    let mut handles = vec![];

    for _ in 0..10 {
        let counter = Arc::clone(&counter);
        handles.push(thread::spawn(move || {
            counter.fetch_add(1, Ordering::SeqCst);
        }));
    }

    for h in handles { h.join().unwrap(); }
    println!("{}", counter.load(Ordering::SeqCst));  // 10
}
```

CHAPTER 19

Modules & Packages

Module System Overview

- **Package** — A Cargo feature that lets you build, test, and share crates. Contains a Cargo.toml.
- **Crate** — A binary or library. The crate root is src/main.rs (binary) or src/lib.rs (library).
- **Module** — Organizes code within a crate. Controls privacy (pub/private).
- **Path** — A way to name an item (struct, function, module) in the module tree.

Defining Modules

Modules

```
// In src/lib.rs
mod front_of_house {
    pub mod hosting {
        pub fn add_to_waitlist() {}
        fn seat_at_table() {}  // Private
    }

    mod serving {  // Private module
        fn take_order() {}
        fn serve_order() {}
    }
}

pub fn eat_at_restaurant() {
    // Absolute path
    crate::front_of_house::hosting::add_to_waitlist();

    // Relative path
    front_of_house::hosting::add_to_waitlist();
}
```

File-Based Modules

File-Based Modules

```
// Project structure:
// src/
//   main.rs
//   lib.rs
//   garden.rs          OR  garden/mod.rs
//   garden/
//     vegetables.rs

// src/lib.rs
pub mod garden;  // Loads from garden.rs or garden/mod.rs

// src/garden.rs
pub mod vegetables;  // Loads from garden/vegetables.rs

// src/garden/vegetables.rs
pub struct Asparagus {
    pub name: String,
}
```

use and Paths

use Statements

```
use std::collections::HashMap;
use std::io::{self, Write};        // Multiple items
use std::collections::*;           // Glob import (use sparingly)

// Re-export
pub use crate::front_of_house::hosting;

// Aliasing
use std::fmt::Result;
use std::io::Result as IoResult;

// Nested paths
use std::{cmp::Ordering, io};
```

Visibility (pub)

Visibility

```
mod my_module {
    pub struct PublicStruct {
        pub public_field: i32,
        private_field: i32,      // Private by default
    }

    pub fn public_function() {}
    fn private_function() {}     // Private

    // Fine-grained visibility
    pub(crate) fn crate_visible() {}  // Visible in crate
    pub(super) fn parent_visible() {} // Visible to parent
}
```

CHAPTER 20

Testing

Unit Tests

Unit Tests

```
// Tests go in the same file, in a tests module
pub fn add(a: i32, b: i32) -> i32 {
    a + b
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_add() {
        assert_eq!(add(2, 3), 5);
    }

    #[test]
    fn test_add_negative() {
        assert_eq!(add(-1, 1), 0);
    }

    #[test]
    #[should_panic(expected = "overflow")]
    fn test_overflow() {
        panic!("overflow");
    }

    #[test]
    fn test_result() -> Result<(), String> {
        if add(2, 2) == 4 {
            Ok(())
        } else {
            Err(String::from("two plus two is not four"))
        }
    }

    #[test]
    #[ignore]  // Skip this test normally
    fn expensive_test() {
        // Long running test
    }
}
```

Assertion Macros

Assertions

```
assert!(expression);                          // Panics if false
assert_eq!(left, right);                      // Panics if not equal
assert_ne!(left, right);                      // Panics if equal
assert!(val, "Custom message: {}", val);      // With message
assert_eq!(a, b, "{} != {}", a, b);           // With message
debug_assert!(expression);                    // Only in debug builds
```

Integration Tests

Integration Tests

```
// tests/integration_test.rs (separate file in tests/ directory)
use my_crate;  // Import the crate being tested

#[test]
fn it_adds_two() {
    assert_eq!(my_crate::add(2, 2), 4);
}
```

Running Tests

Test Commands

```
cargo test                          # Run all tests
cargo test test_add                 # Run tests matching name
cargo test -- --test-threads=1      # Single-threaded
cargo test -- --show-output         # Show println output
cargo test -- --ignored             # Run ignored tests
cargo test --doc                    # Run doc tests only
```

CHAPTER 21

Traits (Advanced)

Associated Types

Associated Types

```
// Associated type in a trait
trait Iterator {
    type Item;  // Associated type
    fn next(&mut self) -> Option<Self::Item>;
}

// vs generics:
// trait Iterator<T> { fn next(&mut self) -> Option<T>; }
// With associated types, you implement ONCE per type.
// With generics, you could implement multiple times.
```

Operator Overloading

Operator Overloading

```
use std::ops::Add;

#[derive(Debug, Clone, Copy)]
struct Point {
    x: f64,
    y: f64,
}

impl Add for Point {
    type Output = Point;

    fn add(self, other: Point) -> Point {
        Point {
            x: self.x + other.x,
            y: self.y + other.y,
        }
    }
}

fn main() {
    let p1 = Point { x: 1.0, y: 0.0 };
    let p2 = Point { x: 2.0, y: 3.0 };
    let p3 = p1 + p2;  // Uses our Add implementation
    println!("{:?}", p3);
}
```

Supertraits

Supertraits

```
use std::fmt;

// OutlinePrint requires Display (supertrait)
trait OutlinePrint: fmt::Display {
    fn outline_print(&self) {
        let output = self.to_string();  // Uses Display
        let len = output.len();
        println!("{}", "*".repeat(len + 4));
        println!("* {} *", output);
        println!("{}", "*".repeat(len + 4));
    }
}
```

Newtype Pattern

Newtype Pattern

```
// Implement external traits on external types using newtype
struct Wrapper(Vec<String>);

impl fmt::Display for Wrapper {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        write!(f, "[{}]", self.0.join(", "))
    }
}

// Access inner value with .0 or implement Deref
impl std::ops::Deref for Wrapper {
    type Target = Vec<String>;
    fn deref(&self) -> &Vec<String> {
        &self.0
    }
}
```

CHAPTER 22 Unsafe Rust

Unsafe Rust gives you five superpowers that safe Rust doesn't allow. Use it when you need to do things the compiler can't verify, but you must manually ensure safety.

Unsafe Superpowers

- Dereference a raw pointer
- Call an unsafe function or method
- Access or modify a mutable static variable
- Implement an unsafe trait
- Access fields of unions

Raw Pointers

Raw Pointers

```
fn main() {
    let mut num = 5;

    // Creating raw pointers is safe
    let r1 = &num as *const i32;   // Immutable raw pointer
    let r2 = &mut num as *mut i32; // Mutable raw pointer

    // Dereferencing requires unsafe
    unsafe {
        println!("r1: {}", *r1);
        println!("r2: {}", *r2);
    }
}
```

Calling Unsafe Functions

Unsafe Functions & FFI

```
unsafe fn dangerous() {
    // Unsafe code here
}

fn main() {
    unsafe {
        dangerous();
    }
}

// FFI (Foreign Function Interface)
extern "C" {
    fn abs(input: i32) -> i32;
}

fn main() {
    unsafe {
        println!("abs(-3) = {}", abs(-3));
    }
}
```

NOTE

Unsafe does NOT turn off the borrow checker. It only gives you the five superpowers listed above. Keep unsafe blocks as small as possible and wrap them in safe abstractions.

CHAPTER 23

Macros

Declarative Macros (macro_rules!)

Declarative Macros

```
// Simple macro
macro_rules! say_hello {
    () => {
        println!("Hello!");
    };
}

// Macro with arguments
macro_rules! create_function {
    ($func_name:ident) => {
        fn $func_name() {
            println!("Called {:?}()", stringify!($func_name));
        }
    };
}

// vec!-like macro
macro_rules! my_vec {
    // Match: my_vec![1, 2, 3]
    ( $( $x:expr ),* ) => {
        {
            let mut temp = Vec::new();
            $(
                temp.push($x);
            )*
            temp
        }
    };
}

fn main() {
    say_hello!();
    create_function!(foo);
    foo();
    let v = my_vec![1, 2, 3];
}
```

Macro Fragment Types

Fragment	Description	Example Match
`$x:expr`	Expression	1 + 2, foo()
`$x:ident`	Identifier	my_var, String

`$x:ty`	Type	i32, Vec<String>
`$x:pat`	Pattern	Some(x), _
`$x:stmt`	Statement	let x = 1
`$x:block`	Block	{ ... }
`$x:item`	Item	fn foo() {}
`$x:path`	Path	std::io::Error
`$x:tt`	Token tree	Any single token
`$x:literal`	Literal	42, "hello"

Procedural Macros (Overview)

Procedural macros accept Rust code as input, operate on that code, and produce Rust code as output. There are three kinds:

- **Custom derive macros** — #[derive(MyTrait)] automatically implements a trait.
- **Attribute-like macros** — #[my_attribute] on functions, structs, etc.
- **Function-like macros** — my_macro!(...) that look like function calls but operate on tokens.

Procedural Macro Usage

```
// Example: derive macro usage
#[derive(Debug, Serialize, Deserialize)]  // Serde macros
struct Config {
    name: String,
    value: i32,
}

// Example: attribute macro usage
#[tokio::main]  // Tokio async runtime
async fn main() {
    // async code here
}
```

CHAPTER 24

Async / Await

Async Basics

Async Basics

```rust
// async functions return a Future
async fn fetch_data() -> String {
    // Simulate async work
    String::from("data")
}

// .await pauses until the future completes
async fn process() {
    let data = fetch_data().await;
    println!("Got: {}", data);
}

// You need a runtime to execute async code
// Most popular: tokio
// Cargo.toml: tokio = { version = "1", features = ["full"] }

#[tokio::main]
async fn main() {
    process().await;
}
```

Concurrent Tasks

Concurrent Tasks

```
use tokio;

#[tokio::main]
async fn main() {
    // Spawn concurrent tasks
    let task1 = tokio::spawn(async {
        // async work
        42
    });

    let task2 = tokio::spawn(async {
        // async work
        "hello"
    });

    // Wait for both
    let (result1, result2) = tokio::join!(task1, task2);
    println!("{:?}, {:?}", result1, result2);

    // Select: wait for first to complete
    tokio::select! {
        val = async { 1 } => println!("first: {}", val),
        val = async { 2 } => println!("second: {}", val),
    }
}
```

Async Streams

Async Streams

```
use tokio_stream::StreamExt;  // tokio-stream crate

async fn example() {
    let mut stream = tokio_stream::iter(vec![1, 2, 3]);

    while let Some(value) = stream.next().await {
        println!("{}", value);
    }
}
```

CHAPTER 25 Standard Library Types

String Formatting

String Formatting

```
fn main() {
    // println! and format! macros
    println!("Hello {}!", "world");           // Positional
    println!("{0} is {1} and {0}", "Rust", "fun"); // Indexed
    println!("{name} = {value}", name="x", value=10);

    // Formatting
    println!("{:>10}", "right");     // Right-align in 10 chars
    println!("{:<10}", "left");      // Left-align
    println!("{:^10}", "center");    // Center
    println!("{:0>5}", 42);          // Zero-pad: 00042
    println!("{:.2}", 3.14159);      // 2 decimal places: 3.14
    println!("{:b}", 42);            // Binary: 101010
    println!("{:o}", 42);            // Octal: 52
    println!("{:x}", 255);           // Hex lowercase: ff
    println!("{:X}", 255);           // Hex uppercase: FF
    println!("{:e}", 1000.0);        // Scientific: 1e3
    println!("{:#?}", vec![1,2,3]);  // Pretty debug
}
```

Path and PathBuf

Path & PathBuf

```
use std::path::{Path, PathBuf};

fn main() {
    let path = Path::new("/tmp/foo/bar.txt");

    println!("Parent: {:?}", path.parent());
    println!("Filename: {:?}", path.file_name());
    println!("Stem: {:?}", path.file_stem());
    println!("Extension: {:?}", path.extension());
    println!("Exists: {}", path.exists());

    // PathBuf is the owned version (like String vs &str)
    let mut path_buf = PathBuf::from("/tmp");
    path_buf.push("foo");
    path_buf.push("bar.txt");
    path_buf.set_extension("rs");
}
```

Duration and Instant

Time

```
use std::time::{Duration, Instant};

fn main() {
    let duration = Duration::from_secs(5);
    let duration = Duration::from_millis(500);
    let duration = Duration::new(5, 500_000_000); // 5.5 secs

    // Timing code
    let start = Instant::now();
    // ... do work ...
    let elapsed = start.elapsed();
    println!("Took: {:?}", elapsed);
}
```

CHAPTER 26

File I/O

Reading Files

Reading Files

```
use std::fs;
use std::io::{self, BufRead, Read};

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Read entire file to string
    let content = fs::read_to_string("file.txt")?;

    // Read to bytes
    let bytes = fs::read("file.bin")?;

    // Read line by line (buffered)
    let file = fs::File::open("file.txt")?;
    let reader = io::BufReader::new(file);
    for line in reader.lines() {
        println!("{}", line?);
    }

    Ok(())
}
```

Writing Files

Writing Files

```rust
use std::fs;
use std::io::Write;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    // Write string to file (creates or overwrites)
    fs::write("output.txt", "Hello, World!")?;

    // Write with more control
    let mut file = fs::File::create("output.txt")?;
    file.write_all(b"Hello, World!")?;

    // Append to file
    let mut file = fs::OpenOptions::new()
        .append(true)
        .create(true)
        .open("log.txt")?;
    writeln!(file, "New log entry")?;

    Ok(())
}
```

Filesystem Operations

Filesystem Operations

```rust
use std::fs;

fn main() -> Result<(), Box<dyn std::error::Error>> {
    fs::create_dir("new_dir")?;              // Create directory
    fs::create_dir_all("a/b/c")?;            // Create nested
    fs::remove_file("file.txt")?;            // Delete file
    fs::remove_dir("empty_dir")?;            // Delete empty dir
    fs::remove_dir_all("dir")?;              // Delete dir + contents
    fs::rename("old.txt", "new.txt")?;       // Rename/move
    fs::copy("src.txt", "dst.txt")?;         // Copy file

    // List directory contents
    for entry in fs::read_dir(".")? {
        let entry = entry?;
        println!("{:?} ({:?})", entry.path(), entry.file_type()?);
    }

    // File metadata
    let meta = fs::metadata("file.txt")?;
    println!("Size: {} bytes", meta.len());
    println!("Is file: {}", meta.is_file());
    println!("Is dir: {}", meta.is_dir());

    Ok(())
}
```

CHAPTER 27

Command Line Programs

Command Line Arguments

Command Line Arguments

```
use std::env;

fn main() {
    // Basic argument access
    let args: Vec<String> = env::args().collect();
    println!("Program: {}", args[0]);
    if args.len() > 1 {
        println!("First arg: {}", args[1]);
    }

    // Environment variables
    match env::var("HOME") {
        Ok(val) => println!("HOME = {}", val),
        Err(e) => println!("HOME not set: {}", e),
    }
}
```

Using clap for Argument Parsing

clap Argument Parsing

```
// Cargo.toml: clap = { version = "4", features = ["derive"] }

use clap::Parser;

#[derive(Parser)]
#[command(name = "myapp", about = "A cool CLI tool")]
struct Args {
    /// Input file path
    #[arg(short, long)]
    input: String,

    /// Output file path
    #[arg(short, long, default_value = "output.txt")]
    output: String,

    /// Verbosity level
    #[arg(short, long, action = clap::ArgAction::Count)]
    verbose: u8,
}

fn main() {
    let args = Args::parse();
    println!("Input: {}", args.input);
    println!("Output: {}", args.output);
    println!("Verbosity: {}", args.verbose);
}
```

Process and Exit Codes

Process Management

```
use std::process;

fn main() {
    // Exit with code
    if some_error_condition() {
        eprintln!("Error: something went wrong");
        process::exit(1);
    }

    // Run external commands
    let output = process::Command::new("ls")
        .arg("-la")
        .output()
        .expect("failed to execute ls");

    println!("stdout: {}", String::from_utf8_lossy(&output.stdout));
    println!("status: {}", output.status);
}

fn some_error_condition() -> bool { false }
```

CHAPTER 28

Web Development Basics

Popular Web Frameworks

- **Axum** — Modern, ergonomic framework by the Tokio team. Great for new projects.
- **Actix-web** — High-performance framework. Very fast benchmarks.
- **Rocket** — Focus on ease of use. Macro-heavy, batteries included.
- **Warp** — Composable filter-based framework.

Axum Example

Axum Web Server

```rust
// Cargo.toml:
// axum = "0.7"
// tokio = { version = "1", features = ["full"] }
// serde = { version = "1", features = ["derive"] }
// serde_json = "1"

use axum::{
    routing::{get, post},
    Router, Json, extract::Path,
};
use serde::{Deserialize, Serialize};

#[derive(Serialize)]
struct User {
    id: u64,
    name: String,
}

#[derive(Deserialize)]
struct CreateUser {
    name: String,
}

async fn hello() -> &'static str {
    "Hello, World!"
}

async fn get_user(Path(id): Path<u64>) -> Json<User> {
    Json(User { id, name: "Alice".to_string() })
}

async fn create_user(Json(payload): Json<CreateUser>)
    -> Json<User>
{
    Json(User { id: 1, name: payload.name })
}

#[tokio::main]
async fn main() {
    let app = Router::new()
        .route("/", get(hello))
        .route("/users/:id", get(get_user))
        .route("/users", post(create_user));

    let listener = tokio::net::TcpListener::bind("0.0.0.0:3000")
        .await.unwrap();
    axum::serve(listener, app).await.unwrap();
}
```

HTTP Client with reqwest

HTTP Client

```rust
// Cargo.toml: reqwest = { version = "0.12", features = ["json"] }

#[tokio::main]
async fn main() -> Result<(), reqwest::Error> {
    // GET request
    let body = reqwest::get("https://httpbin.org/get")
        .await?
        .text()
        .await?;
    println!("{}", body);

    // POST with JSON
    let client = reqwest::Client::new();
    let resp = client.post("https://httpbin.org/post")
        .json(&serde_json::json!({"name": "Rust"}))
        .send()
        .await?;
    println!("Status: {}", resp.status());

    Ok(())
}
```

Serialization with Serde

Serde Serialization

```rust
use serde::{Serialize, Deserialize};

#[derive(Serialize, Deserialize, Debug)]
struct Config {
    name: String,
    version: u32,
    #[serde(default)]
    debug: bool,
    #[serde(rename = "maxRetries")]
    max_retries: u32,
    #[serde(skip_serializing_if = "Option::is_none")]
    optional: Option<String>,
}

fn main() {
    // Serialize to JSON
    let config = Config {
        name: "app".to_string(),
        version: 1,
        debug: false,
        max_retries: 3,
        optional: None,
    };
    let json = serde_json::to_string_pretty(&config).unwrap();
    println!("{}", json);

    // Deserialize from JSON
    let json_str = r#"{"name":"app","version":1,"maxRetries":3}"#;
    let config: Config = serde_json::from_str(json_str).unwrap();
    println!("{:?}", config);
}
```

CHAPTER 29

Best Practices & Idioms

Naming Conventions

Item	Convention	Example
Crates	snake_case	my_crate
Modules	snake_case	my_module
Types (struct, enum)	PascalCase	MyStruct
Traits	PascalCase	Iterator
Functions	snake_case	do_something()
Methods	snake_case	my_method()
Variables	snake_case	my_var
Constants	SCREAMING_SNAKE	MAX_SIZE
Statics	SCREAMING_SNAKE	GLOBAL_COUNT
Type parameters	PascalCase (short)	T, K, V
Lifetimes	short lowercase	'a, 'b

Common Idioms

Common Idioms

```
// 1. Use ? instead of unwrap() in production code
fn good() -> Result<(), Box<dyn std::error::Error>> {
    let content = std::fs::read_to_string("file.txt")?;
    Ok(())
}

// 2. Prefer &str over &String in function params
fn good_param(s: &str) {}       // Accepts &str AND &String
fn bad_param(s: &String) {}     // Only accepts &String

// 3. Use iterators instead of manual loops
let sum: i32 = (1..=100).sum();  // Instead of manual loop

// 4. Use if let for single-pattern matching
if let Some(x) = option { /* use x */ }

// 5. Implement Display for user-facing output
// Implement Debug for developer-facing output

// 6. Use #[derive] when possible
#[derive(Debug, Clone, PartialEq)]
struct MyType { /* ... */ }

// 7. Prefer to_owned() or to_string() over String::from()
let s = "hello".to_owned();

// 8. Use Default trait for structs with many fields
let config = Config {
    name: "custom".to_string(),
    ..Config::default()
};
```

Error Handling Best Practices

- Use **Result** for recoverable errors, **panic!** only for unrecoverable bugs.
- Use the **?** operator to propagate errors upward.
- Define custom error types with **thiserror** for libraries.
- Use **anyhow** for flexible error handling in applications.
- Use **.expect("meaningful message")** instead of **.unwrap()** when you need to panic.
- Add context to errors: **.map_err(|e| format!("reading config: {e}"))**

Performance Tips

- Use **cargo build --release** for production. Debug builds are 10-100x slower.
- Prefer stack allocation (arrays) over heap allocation (Vec) for fixed-size data.
- Use **&str;** and **&[T]** (slices) to avoid unnecessary cloning.
- Use **iterators** -- they are lazy and often optimize better than manual loops.
- Profile before optimizing. Use **cargo flamegraph** or **perf**.

- Use **Cow<str>** when you sometimes need to allocate and sometimes don't.

CHAPTER 30

Appendix: Operator Reference

Arithmetic Operators

Operator	Description	Example	Trait
+	Addition	a + b	Add
-	Subtraction	a - b	Sub
*	Multiplication	a * b	Mul
/	Division	a / b	Div
%	Remainder	a % b	Rem

Comparison Operators

Operator	Description	Trait
==	Equal	PartialEq
!=	Not equal	PartialEq
<	Less than	PartialOrd
>	Greater than	PartialOrd
<=	Less or equal	PartialOrd
>=	Greater or equal	PartialOrd

Logical & Bitwise Operators

Operator	Description
&&	Logical AND (short-circuit)
||	Logical OR (short-circuit)
!	Logical NOT
&	Bitwise AND
|	Bitwise OR
^	Bitwise XOR
<<	Left shift

>>	Right shift

Special Rust Operators & Symbols

Symbol	Meaning
`..`	Range (exclusive end): 0..5
`..=`	Range (inclusive end): 0..=5
`..`	Struct update syntax: ..default
`=>`	Match arm separator
`->`	Function return type
`?`	Error propagation operator
`!`	Macro invocation: println!()
`!`	Never type (diverging function)
_	Wildcard pattern / unused var
`&`	Borrow / reference
`&mut`	Mutable reference
`*`	Dereference
`'a`	Lifetime annotation
`::`	Path separator: std::io::Read
`as`	Type casting: x as f64
`@`	Pattern binding: x @ 1..=5
`#[]`	Attribute: #[derive(Debug)]
`#![]`	Inner attribute: #![allow(...)]

End of Guide

You now have a complete reference for the Rust programming language.

Keep learning at: doc.rust-lang.org • crates.io • rust-lang.org